



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC3533 COMPUTACIÓN DE ALTO RENDIMIENTO
2026-1

Tarea 3: Factorización de Matrices No-Negativas Distribuida

Grupo 25

INTEGRANTES: EMMANUEL NORAMBUENA, DIEGO LIZAMA Y VICENTE RODRÍGUEZ

Fecha de entrega: 18 de Junio, 2026

Contents

1	Introducción	3
1.1	Problema y objetivos	3
1.2	NMF y ALS: descripción general	3
2	Diseño de la Solución Distribuida	4
2.1	Distribución de datos y organización de procesos	4
2.2	Comunicación y operaciones distribuidas	4
2.2.1	$H^T H$ y $W^T W$	4
2.2.2	AH	5
2.2.3	$A^T W$	5
2.2.4	Error de reconstrucción	5
2.3	Algoritmo ALS distribuido	5
2.4	Consideraciones de implementación	6
2.4.1	Regularización del sistema lineal	6
2.4.2	Precisión numérica	6
2.4.3	Threads NumPy	6
3	Metodología Experimental	7
3.1	Entorno de ejecución	7
3.2	Configuración experimental	7
3.3	Métricas de evaluación	7
4	Resultados y Análisis	8
4.1	Validación de correctitud	8
4.2	Escalamiento y rendimiento	8
4.3	Impacto de la configuración de procesos	8
4.4	Perfil de ejecución y cuellos de botella	8
4.5	Experimentos NUMA	9
5	Discusión	10
5.1	Síntesis de hallazgos	10
5.2	Limitaciones observadas	11
5.3	Escalabilidad y trabajo futuro	11

6 Conclusiones

11

1. Introducción

1.1. Problema y objetivos

En muchas aplicaciones científicas e industriales, matrices no-negativas de gran escala no caben en la memoria de un solo proceso. Esta tarea aborda ese problema implementando la Factorización de Matrices No-Negativas (NMF) de forma distribuida, combinando dos niveles de paralelismo: MPI para distribuir la matriz entre procesos a través de una grilla 2D, y múltiples *threads* NumPy para paralelizar el cómputo denso dentro de cada proceso.

El objetivo principal es aproximar una matriz no-negativa $A \in \mathbb{R}^{m \times n}$ como el producto de dos matrices no-negativas:

$$A \approx WH^T, \quad W \in \mathbb{R}^{m \times k}, \quad H \in \mathbb{R}^{n \times k}$$

donde $k \ll \min(m, n)$ es el rango de la factorización. Los objetivos específicos son: (1) implementar correctamente las operaciones distribuidas necesarias, (2) medir el rendimiento en distintas configuraciones de grilla MPI y número de *threads*, y (3) analizar el efecto de la topología de procesos y la afinidad NUMA sobre el tiempo de ejecución.

1.2. NMF y ALS: descripción general

El algoritmo de Mínimos Cuadrados Alternados (ALS) minimiza $\|A - WH^T\|_F^2$ fijando alternativamente uno de los factores y resolviendo por el otro. Partiendo de inicializaciones aleatorias no-negativas de W y H , cada iteración realiza los siguientes pasos:

Actualización de W (con H fijo):

1. Calcular $G_H = H^T H \in \mathbb{R}^{k \times k}$.
2. Calcular $\tilde{W} = AH \in \mathbb{R}^{m \times k}$.
3. Resolver $G_H X = \tilde{W}^T$ para $X \in \mathbb{R}^{k \times m}$.
4. Actualizar $W \leftarrow \max(0, X^T)$.

Actualización de H (con W fijo):

1. Calcular $G_W = W^T W \in \mathbb{R}^{k \times k}$.
2. Calcular $\tilde{H} = A^T W \in \mathbb{R}^{n \times k}$.
3. Resolver $G_W X = \tilde{H}^T$ para $X \in \mathbb{R}^{k \times n}$.
4. Actualizar $H \leftarrow \max(0, X^T)$.

Criterio de convergencia: el algoritmo se detiene cuando el cambio relativo en el error de reconstrucción entre iteraciones consecutivas cae por debajo de una tolerancia ε :

$$\frac{\|A - WH^T\|_F^{(t-1)} - \|A - WH^T\|_F^{(t)}}{\|A - WH^T\|_F^{(t-1)}} < \varepsilon$$

2. Diseño de la Solución Distribuida

2.1. Distribución de datos y organización de procesos

Se organiza un conjunto de $p_r \times p_c$ procesos MPI en una grilla lógica 2D. A cada proceso se le asigna una coordenada (i, j) a partir de su *rank* global r :

$$i = \lfloor r/p_c \rfloor, \quad j = r \bmod p_c$$

Bajo el supuesto de divisibilidad exacta, los datos se distribuyen de la siguiente manera:

- El proceso (i, j) almacena el bloque $A_{ij} \in \mathbb{R}^{(m/p_r) \times (n/p_c)}$ de la matriz A .
- El proceso (i, j) almacena las filas $W_i \in \mathbb{R}^{(m/p_r) \times k}$ de W , correspondientes al rango de filas $[i \cdot m/p_r, (i+1) \cdot m/p_r)$.
- El proceso (i, j) almacena las filas $H_j \in \mathbb{R}^{(n/p_c) \times k}$ de H , correspondientes al rango de columnas $[j \cdot n/p_c, (j+1) \cdot n/p_c)$.

Esta distribución implica que cada proceso posee toda la información local necesaria para calcular su contribución a las operaciones matriciales del algoritmo ALS sin necesidad de redistribuir datos entre iteraciones.

2.2. Comunicación y operaciones distribuidas

Se definen dos comunicadores adicionales al comunicador global mediante `comm.Split`:

```
row_comm = comm.Split(color=i, key=j) # procesos de la misma fila i
col_comm = comm.Split(color=j, key=i) # procesos de la misma columna j
```

Cada operación colectiva se realiza sobre el comunicador apropiado:

2.2.1. $H^T H$ y $W^T W$

Dado que $X^T X = \sum_{\ell} X_{\ell}^T X_{\ell}$, cada proceso debería contribuir con $H_j^T H_j$ (respectivamente $W_i^T W_i$). Sin embargo, en la grilla 2D, H_j es compartida por todos los procesos de la misma columna j , por lo que sumar sobre todos los procesos duplicaría las contribuciones. Para evitarlo, solo los procesos de la primera fila ($i = 0$) aportan a $H^T H$, y solo los de la primera columna ($j = 0$) a $W^T W$; el resto contribuye con ceros. La reducción se realiza con `Allreduce` sobre el comunicador global:

```
def distributed_HtH(Hj, grid):
    local = (Hj.T @ Hj).astype(np.float32) if grid.i == 0 \
        else np.zeros((k, k), dtype=np.float32)
    grid.comm.Allreduce(local, result, op=MPI.SUM)
    return result
```

2.2.2. AH

El proceso (i, j) calcula $A_{ij}H_j$ localmente. Los procesos de la misma fila i acumulan sus contribuciones para obtener:

$$\tilde{W}_i = \sum_{\ell=0}^{p_c-1} A_{i\ell} H_\ell$$

La reducción usa Allreduce sobre row_comm, de modo que todos los procesos de la fila obtienen $\tilde{W}_i$, necesario para la actualización de W_i :

```
def distributed_AH(Aij, Hj, grid):
    local = (Aij @ Hj).astype(np.float32)
    grid.row_comm.Allreduce(local, result, op=MPI.SUM)
    return result
```

2.2.3. $A^\top W$

Análogamente, el proceso (i, j) calcula $A_{ij}^\top W_i$ localmente. Los procesos de la misma columna j acumulan para obtener:

$$\tilde{H}_j = \sum_{\ell=0}^{p_r-1} A_{\ell j}^\top W_\ell$$

La reducción usa Allreduce sobre col_comm:

```
def distributed_ATW(Aij, Wi, grid):
    local = (Aij.T @ Wi).astype(np.float32)
    grid.col_comm.Allreduce(local, result, op=MPI.SUM)
    return result
```

2.2.4. Error de reconstrucción

Cada proceso calcula la norma de Frobenius al cuadrado de su residuo local $A_{ij} - W_i H_j^\top$. La suma global se acumula con Allreduce sobre el comunicador global:

$$\|A - WH^\top\|_F^2 = \sum_{i,j} \|A_{ij} - W_i H_j^\top\|_F^2$$

```
def distributed_error(Aij, Wi, Hj, grid):
    residual = Aij - Wi @ Hj.T
    local_sq = np.array(np.sum(residual ** 2), dtype=np.float64)
    grid.comm.Allreduce(local_sq, global_sq, op=MPI.SUM)
    return np.sqrt(global_sq)
```

2.3. Algoritmo ALS distribuido

El loop principal combina las subrutinas anteriores siguiendo los pasos del algoritmo ALS. Los tiempos de cada operación se miden con `MPI.Wtime()` para el profiling posterior:

```

for it in range(max_iter):

    # --- Actualizar W ---
    GH = distributed_HtH(Hj, grid) # H^T H (Allreduce global)
    W_tilde = distributed_AH(Aij, Hj, grid) # AH (Allreduce row_comm)
    X = solve_regularized(GH, W_tilde.T) # GH X = W_tilde^T
    Wi = np.maximum(0.0, X.T)

    # --- Actualizar H ---
    GW = distributed_WtW(Wi, grid) # W^T W (Allreduce global)
    H_tilde = distributed_ATW(Aij, Wi, grid) # A^T W (Allreduce col_comm)
    X = solve_regularized(GW, H_tilde.T) # GW X = H_tilde^T
    Hj = np.maximum(0.0, X.T)

    # --- Convergencia ---
    err = distributed_error(Aij, Wi, Hj, grid)
    delta = abs(err - prev_err) / prev_err
    if delta < tol:
        break

```

2.4. Consideraciones de implementación

2.4.1. Regularización del sistema lineal

El sistema de ecuaciones normales $GX = B$ puede estar mal condicionado cuando la matriz de Gram G tiene valores propios cercanos a cero. Para garantizar estabilidad numérica se añade una perturbación diagonal:

$$\hat{G} = G + \lambda I, \quad \lambda = 10^{-6} \cdot \max\left(\frac{\text{tr}(G)}{k}, 1\right)$$

Esta regularización escala con la magnitud de G y no afecta la convergencia en la práctica, ya que λ es varios órdenes de magnitud menor que los valores propios dominantes.

2.4.2. Precisión numérica

Las multiplicaciones matriciales densas se realizan en float32 para reducir el uso de memoria y aprovechar el mayor rendimiento BLAS en precisión simple. Los acumuladores de normas y el solve se elevan a float64 para evitar pérdida de precisión en las reducciones globales.

2.4.3. Threads NumPy

El paralelismo intra-proceso se delega enteramente a BLAS/LAPACK a través de NumPy, controlado por la variable de entorno OMP_NUM_THREADS. Esto permite explorar el espacio de configuraciones híbridas MPI+threads sin modificar el código, simplemente variando el número de procesos y -cpus-per-task en los scripts SLURM.

3. Metodología Experimental

3.1. Entorno de ejecución

Los experimentos se ejecutaron en el cluster del curso, partición `hpc-iic3533`, compuesto por dos nodos (*Ventress* y *Yodaxico*), cada uno con 8 CPUs disponibles y 4 GB de RAM por usuario. Los jobs se enviaron mediante SLURM con un tiempo máximo de 15 minutos por ejecución. El entorno de software se configuró con Miniconda usando los paquetes `numpy` y `mpi4py` instalados desde `conda-forge`.

Cada script SLURM especificó explícitamente:

- `-ntasks =  $p_r \times p_c$` : número total de procesos MPI.
- `-cpus-per-task =  $T$` : CPUs por proceso, que determinan los *threads* NumPy vía `OMP_NUM_THREADS`.
- `-mem=4G`: límite de memoria por nodo.
- `-ntasks-per-socket`: afinidad NUMA de los procesos.

3.2. Configuración experimental

Se evaluaron todas las combinaciones de grilla MPI (p_r, p_c) y número de *threads* T indicadas en el enunciado, con parámetros fijos $m = 20,000$, $n = 5,000$, $k = 20$, `seed=42` y un máximo de 20 iteraciones ALS. La Tabla 1 resume las 18 configuraciones oficiales más las dos variantes NUMA.

Table 1: Configuraciones experimentales evaluadas.

Config	p_r	p_c	$p_r \times p_c$	T	Nodos	CPUs totales
0	1	1	1	1, 2, 4	1, 1, 1	1, 2, 4
1	2	1	2	1, 2, 4	1, 1, 1	2, 4, 8
2	1	2	2	1, 2, 4	1, 1, 1	2, 4, 8
3	2	2	4	1, 2, 4	1, 1, 2	4, 8, 16
4	4	1	4	1, 2, 4	1, 1, 2	4, 8, 16
5	1	4	4	1, 2, 4	1, 1, 2	4, 8, 16
NUMA-a	2	1	2	2	1	4
NUMA-b	2	1	2	2	1	4

Las dos variantes NUMA corresponden a la configuración 2×1 , $T = 2$, ejecutadas con `-ntasks-per-socket=1` (un proceso por socket) y `-ntasks-per-socket=2` (ambos procesos en el mismo socket), manteniendo el resto de parámetros idénticos.

3.3. Métricas de evaluación

Para cada configuración se registraron las siguientes métricas:

- **Tiempo total** (`elapsed_sec`): medido con `MPI.Wtime()` con barreras `comm.Barrier()` antes y después del loop ALS.
- **Tiempo por iteración** (`time_per_iter`): permite comparar configuraciones independientemente del número de iteraciones ejecutadas.

- **Error relativo:** $\|A - WH^T\|_F / \|A\|_F$, indicador de calidad de la factorización.
- **Normas de Frobenius** de A , W y H : usadas para verificar reproducibilidad entre configuraciones.
- **Perfil por operación:** tiempo acumulado en cada subrutina (`hth`, `ah`, `wtw`, `atw`, `solve_w`, `solve_h`, `error`) para identificar cuellos de botella.

4. Resultados y Análisis

4.1. Validación de correctitud

La norma de Frobenius de A es idéntica en todas las configuraciones ($\|A\|_F = 50,995.78$), lo que confirma que la generación y partición de la matriz es reproducible independientemente de la grilla MPI utilizada. El error relativo final converge al mismo valor (≈ 0.0265) en todos los experimentos, y las normas de W y H se mantienen en rangos consistentes ($\|W\|_F \approx 22.7$, $\|H\|_F \approx 4700$).

Las pequeñas variaciones en $\|W\|_F$ y $\|H\|_F$ entre configuraciones se deben al orden no determinista de las operaciones en punto flotante al acumular contribuciones de distintos procesos. Sin embargo, dado que el error relativo converge al mismo valor, estas diferencias no afectan la calidad de la factorización.

4.2. Escalamiento y rendimiento

La Tabla 2 muestra el tiempo total y por iteración para todas las configuraciones.

La configuración **2×2 con $T = 4$** obtiene el menor tiempo total (4.05 s), una mejora de 4.9× respecto al caso base (1×1 , $T = 1$, 19.93 s). Se observa que aumentar T de 1 a 4 *threads* reduce consistentemente el tiempo en todas las grillas, con ganancias entre 1.4× y 2.0×. El caso 1×1 con $T = 2$ es la única excepción donde más *threads* no mejoran el tiempo, posiblemente por sobrecarga de sincronización en un único proceso con cómputo dominado por el error.

4.3. Impacto de la configuración de procesos

Las configuraciones 3 (2×2), 4 (4×1) y 5 (1×4) utilizan el mismo número total de procesos MPI ($p_r \times p_c = 4$), lo que permite aislar el efecto de la forma de la grilla.

La grilla 2×2 es consistentemente la más rápida. Esto se explica porque la matriz A tiene dimensiones $20,000 \times 5,000$ (relación 4:1 entre filas y columnas). Distribuir tanto en filas como en columnas produce bloques más cuadrados, equilibrando el volumen de comunicación en las operaciones AH (sobre `row_comm`) y $A^T W$ (sobre `col_comm`). En cambio, la grilla 1×4 genera bloques muy anchos que sobrecargan `col_comm`, y 4×1 genera bloques muy altos que sobrecargan `row_comm`.

4.4. Perfil de ejecución y cuellos de botella

La Tabla 4 muestra el perfil de tiempos para la configuración 1×1 con $T = 1$, donde el cómputo está completamente serializado y los porcentajes son más interpretables.

El principal cuello de botella es el **cálculo del error de reconstrucción**, que consume el 67.9% del tiempo. Esta operación requiere materializar el residuo $A_{ij} - W_i H_j^T$ completo en memoria para cada bloque local, con costo $O(m \cdot n \cdot k / (p_r \cdot p_c))$ por proceso por iteración. Las multiplicaciones $A^T W$ y AH son el

p_r	p_c	T	Total (s)	Por iteración (s)
1	1	1	19.93	0.996
1	1	2	21.67	1.084
1	1	4	13.83	0.692
2	1	1	16.30	0.815
2	1	2	10.80	0.540
2	1	4	7.41	0.370
1	2	1	17.16	0.858
1	2	2	9.46	0.473
1	2	4	7.62	0.381
2	2	1	9.02	0.451
2	2	2	5.52	0.276
2	2	4	4.05	0.202
4	1	1	8.92	0.446
4	1	2	5.93	0.297
4	1	4	4.19	0.210
1	4	1	9.13	0.456
1	4	2	5.77	0.288
1	4	4	4.54	0.227

Table 2: Tiempo total (s) y tiempo por iteración (s/iter) por configuración.

p_r	p_c	$T = 1$ (s)	$T = 2$ (s)	$T = 4$ (s)
2	2	9.02	5.52	4.05
4	1	8.92	5.93	4.19
1	4	9.13	5.77	4.54

Table 3: Comparación de grillas con $p_r \times p_c = 4$ procesos.

segundo y tercer factor más costoso ($\approx 30\%$ combinadas), pues involucran tanto cómputo denso como comunicación MPI. Los solves y los productos de Gram ($k \times k$) son negligibles dado que $k = 20 \ll m, n$.

Este patrón se mantiene en todas las configuraciones: al aumentar el número de procesos, el tiempo en **error**, **ah** y **atw** escala aproximadamente como $1/(p_r \cdot p_c)$, mientras que la sobrecarga MPI crece moderadamente.

4.5. Experimentos NUMA

Para la configuración 2×1 con $T = 2$ se ejecutaron dos variantes con distinta afinidad NUMA:

Ejecutar un proceso por socket resulta un **18% más rápido**. Cuando ambos procesos comparten el mismo socket, compiten por el controlador de memoria NUMA local, generando contención en el ancho de banda. Con un proceso por socket, cada uno accede exclusivamente a su banco de memoria, eliminando esa competencia. Este resultado ilustra que la localidad de memoria puede tener un impacto significativo

Operación	Tiempo (s)	%
Error ($\ A - WH^T\ _F$)	13.37	67.9%
$A^T W$	3.62	18.4%
AH	2.40	12.2%
Solve W	0.24	1.2%
$H^T H$	0.004	0.02%
$W^T W$	0.010	0.05%
Solve H	0.040	0.20%
Total	19.68	100%

Table 4: Perfil de tiempos para la configuración 1×1 , $T = 1$ (20 iteraciones).

Variante	Total (s)	Por iteración (s)
Mismo socket (-ntasks-per-socket=2)	10.75	0.538
Sockets distintos (-ntasks-per-socket=1)	8.81	0.440
Diferencia	-18%	

Table 5: Efecto de la afinidad NUMA ($p_r = 2$, $p_c = 1$, $T = 2$).

incluso cuando el número total de CPUs es idéntico.

5. Discusión

5.1. Síntesis de hallazgos

Los resultados obtenidos permiten concluir que la implementación distribuida de NMF fue capaz de ejecutar correctamente el algoritmo ALS sobre una grilla bidimensional de procesos MPI, manteniendo resultados numéricamente consistentes entre las distintas configuraciones evaluadas. Los valores de error relativo obtenidos fueron similares en todos los experimentos, lo que valida tanto la estrategia de distribución de datos como las operaciones colectivas implementadas para construir las matrices de Gram, calcular los productos distribuidos y evaluar el error global.

Desde el punto de vista del rendimiento, se observó una reducción significativa del tiempo de ejecución al incrementar el nivel de paralelismo. Tanto el aumento del número de procesos MPI como la utilización de múltiples hilos por proceso contribuyeron a disminuir el tiempo total requerido por el algoritmo. Sin embargo, las mejoras obtenidas no fueron estrictamente lineales, evidenciando la presencia de costos asociados a comunicación, sincronización y acceso compartido a memoria.

Asimismo, los experimentos mostraron que la forma de la grilla de procesos influye sobre el desempeño de la aplicación. Aun utilizando la misma cantidad total de procesos, distintas distribuciones de la matriz generan patrones de comunicación diferentes, afectando el tiempo requerido por algunas operaciones colectivas. Esto confirma que, en aplicaciones distribuidas, la organización de los procesos puede ser tan relevante como la cantidad de recursos utilizados.

5.2. Limitaciones observadas

A pesar de los resultados obtenidos, la implementación presenta algunas limitaciones que deben ser consideradas al interpretar los experimentos.

En primer lugar, el tamaño de la matriz utilizado estuvo condicionado por los recursos disponibles en el cluster y por las restricciones de tiempo de ejecución impuestas por la partición utilizada. Como consecuencia, no fue posible evaluar configuraciones de mayor escala que permitieran observar con mayor claridad el comportamiento asintótico del algoritmo.

Por otra parte, aunque la implementación distribuye exitosamente las operaciones principales de ALS, ciertas etapas requieren reducciones globales y sincronizaciones entre procesos que introducen costos inevitables de comunicación. A medida que aumenta el número de procesos, estos costos representan una fracción cada vez más importante del tiempo total de ejecución, limitando la eficiencia paralela alcanzable.

Finalmente, el estudio de afinidad NUMA se realizó sobre una cantidad acotada de configuraciones, por lo que sus resultados deben interpretarse principalmente como una validación exploratoria del impacto que puede tener la ubicación física de los procesos sobre el rendimiento de aplicaciones híbridas MPI y multithreading.

5.3. Escalabilidad y trabajo futuro

Los resultados sugieren que la estrategia de paralelización utilizada posee potencial para escalar hacia problemas de mayor tamaño. La distribución bidimensional de la matriz permite repartir tanto almacenamiento como carga computacional entre múltiples procesos, manteniendo una estructura de comunicación relativamente eficiente para las operaciones requeridas por ALS.

Como trabajo futuro, sería interesante evaluar el comportamiento de la implementación utilizando matrices significativamente más grandes y una mayor cantidad de nodos de cómputo. Esto permitiría estudiar con mayor profundidad el balance entre costos de comunicación y cómputo, así como identificar los límites prácticos de escalabilidad de la solución.

Adicionalmente, podrían explorarse optimizaciones orientadas a reducir la frecuencia de sincronizaciones globales, mejorar la superposición entre comunicación y cómputo, o incorporar bibliotecas especializadas de álgebra lineal distribuida. Estas mejoras podrían contribuir a aumentar la eficiencia paralela y aprovechar de mejor manera los recursos disponibles en arquitecturas de alto rendimiento.

6. Conclusiones

En este trabajo se desarrolló una implementación distribuida del algoritmo NMF utilizando MPI sobre una grilla bidimensional de procesos. Los resultados obtenidos muestran que la solución implementada es correcta desde el punto de vista numérico, manteniendo errores relativos consistentes entre las distintas configuraciones evaluadas.

Los experimentos también evidencian que la paralelización permite reducir significativamente los tiempos de ejecución. Sin embargo, las mejoras observadas no son completamente lineales debido a los costos de comunicación y sincronización que aparecen al distribuir el trabajo entre múltiples procesos. Además, se observó que la forma de la grilla y la ubicación de los procesos pueden influir en el rendimiento final de la aplicación.

En conjunto, los resultados validan el diseño propuesto y muestran de manera práctica uno de los

principales desafíos de la computación de alto rendimiento: encontrar un equilibrio adecuado entre el trabajo computacional realizado por cada proceso y el costo de coordinar la información distribuida entre ellos.